

VHDL'i Araştır

Kripto Algoritmalarını VHDL'e Implement Etmek

LFSR Nedir

Adım 1: LFSR (Linear Feedback Shift Register) Nedir?

- Tanım:** LFSR, çıkış biti (ya da bitleri) önceki durumunun doğrusal bir fonksiyonu olan bir kaydırma yazmacıdır. Genellikle bu doğrusal fonksiyon, yazmacın belirli bitlerinin **Özel VEYA (XOR)** işlemine tabi tutulmasıyla elde edilen geri besleme bitidir.
- Temel Bileşenler:**
 - Kaydırma Yazmacı (Shift Register):** D-tipi flip-flop'lardan oluşan ve her saat vuruşunda içeriği bir bit kaydıran dizi.
 - Geri Besleme (Feedback):** Yazmacın belirli bitlerinin (bunlara **"Tap"** denir) XOR işleminden geçirilmesiyle elde edilen ve yazmacın girişine beslenen yeni bit.
 - Tohum (Seed):** LFSR'nin ilk başlangıç değeri (tüm bitlerin sıfır olmadığı bir değer olmalıdır, aksi halde çalışmayı durdurur).
- LFSR Mimarileri:** İki ana mimarisi vardır:
 - Fibonacci LFSR (Dıştan Geri Besleme):** XOR kapıları yazmacın dışında bulunur ve çıkışı ilk flip-flop'un girişine beslenir.
 - Galois LFSR (İçten Geri Besleme):** XOR kapıları ardışık flip-flop'lar arasına yerleştirilmiştir. Donanımda (FPGA'de) genellikle daha verimli olduğu kabul edilir çünkü XOR gecikmesi bir flip-flop aşamasıyla sınırlıdır. **Galois mimarisine odaklanmanız faydalı olabilir.**
- Maksimal Uzunluk:** N bit uzunluğundaki bir LFSR, başlangıç durumu hariç $2^N - 1$ farklı durumu dolaşabilir. Bu durumlara "yalancı rastgele dizi" (m-sequence) denir. Bu maksimum uzunluğa ulaşmak için kullanılan XOR bağlantılarını belirleyen polinoma **"İlkel Polinom (Primitive Polynomial)"** denir. LFSR'nin uzunluğunu seçtikten sonra, o uzunluğa karşılık gelen ilkel polinomu bulmanız gerekecektir.

Adım 2: VHDL'de LFSR Implementasyonu

VHDL öğrenimi için bu mükemmel bir uygulamadır, çünkü ardışıl (sequential) devre tasarımının temellerini içerir.

- Mimariyi Seçin:** Galois mimarisi genellikle daha hızlı olduğu için tercih edilir.
- Uzunluk ve Polinom Seçimi:** Basit bir örnek için 8-bit LFSR'yi seçebilirsiniz. 8-bit için ilkel bir polinom örneği: $x^8 + x^4 + x^3 + x^2 + 1$. Bu, 8., 4., 3. ve 2. bitlerin XOR'lanıp yeni bir bit üretileceği anlamına gelir (Genellikle x yazmacın -inci biti olarak yorumlanır).
- VHDL Kodu Temelleri:**
 - Entity Tanımı:** Saat (clk), reset (rst) ve çıkış (lfsr_out) portlarını tanımlayın. Geri besleme bitlerinin konumlarını (tap'leri) belirlemek için bir **Generic** (Jenerik) kullanabilirsiniz, bu kodunuzu esnek yapar. VHDL

```
entity lfsr is
    generic ( N : integer := 8 ); -- LFSR uzunluğu
```

```

    port (
        clk      : in  std_logic;
        rst      : in  std_logic;
        lfsr_out: out std_logic_vector(N-1 downto 0)
    );
end entity lfsr;

```

- **Architecture (Mimari):** Bir process kullanarak ardışıl mantığı (shift register) uygulayın.
 - Bir **internal signal** (dahili sinyal) tanımlayın: `signal lfsr_reg : std_logic_vector(N-1 downto 0);`
 - `process(clk, rst)` bloğu içinde:
 - **Reset Durumu:** `if rst = '1' then lfsr_reg <= "10000000";` (Örnek tohum değeri)
 - **Saat Vurumu:** `elsif rising_edge(clk) then`
 - **Geri Besleme Hesabı:** Seçtiğiniz polinoma göre XOR işlemini yapın. (Örnek 8-bit için $x^8 + x^4 + x^3 + x^2 + 1$) VHDL

```

feedback <= lfsr_reg(N-1) xor lfsr_reg(3) xor lfsr_reg(2) xor
lfsr_reg(1); -- lfsr_reg(N-1) en son biti temsil edebilir

```

- **Kaydırma İşlemi:** Geri besleme bitini yazmacın ilk bitine yerleştirip diğer bitleri kaydırın. VHDL

```

lfsr_reg <= feedback & lfsr_reg(N-1 downto 1); -- (Fibonacci için örnek)

```

VEYA (Galois için içten XOR mantığı biraz daha farklıdır, örnekleri inceleyin).

Adım 3: Kripto Algoritmalarıyla Bağlantı Kurma

LFSR'yi öğrenirken hocanızın asıl istediği olan "kripto" kısmını da düşünmeye başlayın:

- **Stream Şifreleyiciler (Stream Ciphers):** LFSR'ler, özellikle **donanım tabanlı** stream şifreleyicilerin (örneğin, A5/1 algoritması, GSM'de kullanılır) ana bileşenleridir. Bir LFSR'nin çıktısı olan sözde rastgele bit dizisi (keystream), düz metinle XOR'lanarak şifreli metin elde edilir.
- **Görevinizi Geliştirme:**
 1. **Basit Bir keystream Üretin:** Tasarladığınız LFSR'nin bir saat vuruşunda ürettiği bit dizisini (keystream) alın.
 2. **Şifreleme Simülasyonu:** Basitçe, dışarıdan gelen bir giriş biti dizisini (düz metin) bu keystream ile XOR işlemine tabi tutarak şifreli metin üreten bir VHDL modülü daha ekleyin.

Önerilen Yapılacaklar Listesi

1. **Teoriyi Anlayın:** LFSR'nin ne olduğunu, Fibonacci ve Galois mimarileri arasındaki farkları ve neden ilkel polinom kullandığımızı (maksimum periyot için) öğrenin.
2. **8-Bit Galois LFSR VHDL Kodu:** İnternetteki örnekleri inceleyerek (özellikle Galois mimarisi) 8-bit'lik bir LFSR'yi VHDL'de kodlayın. İlkel bir polinom kullanmaya dikkat edin.
3. **Simülasyon (Testbench):** Yaptığınız LFSR'yi bir **Testbench (TB)** ile simüle edin.
 - Reset sinyali uygulayın.
 - Bir başlangıç tohumu (seed) yükleyin.

- LFSR'nin periyodunu ($2^8 - 1 = 255$ saat vuruşu olmalı) kontrol edin.
4. **Raporlama:** Hocanıza sunmak üzere: LFSR'nin tanımı, mimari seçimi (Fibonacci/Galois), kullanılan polinom ve VHDL kodunuz ile simülasyon sonuçlarını içeren kısa bir rapor hazırlayın.

Hocaya söylenecek

Hocam merhabalar ben LFSR için giriş araştırmamı yaptım. Youtubedaki videolar ve yapay zeka ile araştırma yaptım. Linear Feedback Shift register temelinde flip flop ile sinyali kaydıldıktan sonra belli bir polinoma göre xor kapısı ile binary sayımızı değiştiren ve $(2^n) - 1$ den sonra başa dönen bir geri besleme. Cryptography de rastgele sayı oluşturmak için ve ya rastgele bir sinyal için polinom temelli bir geri besleme. Anladığım kadarıyla tek bir LFSR döngüsünü sisteminin çözülebileceği için çift veya daha fazla LFSR döngüsü ve farklı devir zamanlarına sahip olan döngüleri xor layarak kırılması neredeyse imkansız sistemler oluşturulmak için kullanılabilir ya da bir metni şifrelemek için belli bir polinom kullanıp cevabı alacak yerde de tam tersiyle çözümlmek mümkün olabilir. Gördüğüm kadarıyla iki farklı polinom tipi var Fibonacci ve Galois mimarileri. VHDL ile bir kod şuanda yazmadım hocam.

KAYNAK

<https://www.youtube.com/watch?v=Ks1pw1X22y4>

[https://ieeexplore.ieee.org/abstract/document/8821227?](https://ieeexplore.ieee.org/abstract/document/8821227?casa_token=rj6JRtJGXFAAAAAA:b6wXSh9ITaEE7sso3JZus9CsPLZURZ0P1DSF_izjycc2E5quwYo9jacuJj4dQixRVkd_m-Y)

[cuJj4dQixRVkd_m-Y](https://ieeexplore.ieee.org/abstract/document/8821227?casa_token=rj6JRtJGXFAAAAAA:b6wXSh9ITaEE7sso3JZus9CsPLZURZ0P1DSF_izjycc2E5quwYo9jacuJj4dQixRVkd_m-Y)